HelixQA Vision & Chat Models — Comprehensive Research

Date: 2026-03-31 **Author**: AI Research (Claude Opus 4.6) **Purpose**: Guide model selection for each HelixQA autonomous QA pipeline phase

---

Table of Contents

1. Executive Summary
2. Phase Requirements Analysis
3. Vision Models for Navigation (Execute/Curiosity)
4. Vision Models for Analysis (Analyze Phase)
5. Chat Models for Planning (Learn/Plan)
6. Local & Open-Source Models
7. Bridged CLI Models
8. Specialized Vision APIs
9. Cost Analysis
10. Recommended Configuration
11. Revolutionary Ideas
12. Sources

---

1. Executive Summary

HelixQA's autonomous QA pipeline has 5 distinct phases, each with different model requirements. Using a single model for all phases is suboptimal — **navigation needs fast JSON-producing models**, while **analysis needs rich descriptive models**, and **planning needs strong reasoning models**.

Key Findings

Phase	Best Model Type	Top Recommendation	Cost
Learn	Large-context chat	Gemini 2.5 Flash (1M context)	Free tier
Plan	Strong reasoning	Claude Sonnet 4 / Gemini 2.5 Pro	$0.003-2/1M tokens
Execute/Curiosity	Fast JSON vision	Gemini 2.5 Flash / Kimi K2.5	$0.10-0.60/1M tokens
Analyze	Rich vision description	Astica Vision 2.5 / GPT-4o	$0.001-5/1M tokens

Critical Insight

Astica.AI excels at image analysis (OCR, object detection, captioning) but **cannot produce structured JSON navigation commands**. It must be used ONLY for the Analyze phase, NOT for Execute/Curiosity. This is the root cause of the "stuck on search screen" issue.

2. Phase Requirements Analysis

Phase 1: Learn (Knowledge Base Loading)

- **Task**: Load project documentation, parse screens, extract endpoints
- **Model needs**: Large context window (>100K tokens), fast processing, text-only
- **Vision needed**: No
- **JSON output needed**: No
- **Speed priority**: Medium (runs once per session)

Phase 2: Plan (Test Generation)

- **Task**: Generate test cases from knowledge base context
- **Model needs**: Strong reasoning, structured output (test case format)
- **Vision needed**: No
- **JSON output needed**: Yes (test plan structure)
- **Speed priority**: Low (runs once per session)

Phase 3: Execute (Test Execution)

- **Task**: Take screenshots, analyze UI state, produce navigation actions
- **Model needs**: Vision + JSON output + fast response
- **Vision needed**: YES (critical)
- **JSON output needed**: YES (critical — must produce `[{"type":"dpad_down","reason":"..."}]`)
- **Speed priority**: HIGH (called per test step, 10-50 times per session)

Phase 3.5: Curiosity (Free Exploration)

- **Task**: Same as Execute but open-ended exploration
- **Model needs**: Same as Execute + creativity/exploration instinct
- **Vision needed**: YES
- **JSON output needed**: YES
- **Speed priority**: HIGH (called 50+ times per session)

Phase 4: Analyze (Issue Detection)

- **Task**: Analyze screenshots for visual bugs, UI issues, accessibility problems
- **Model needs**: Rich image description, detail detection, OCR
- **Vision needed**: YES (critical)

- **JSON output needed**: No (natural language descriptions preferred)
- **Speed priority**: Low (runs once on selected screenshots)

---

3. Vision Models for Navigation (Execute/Curiosity)

These models must produce **valid JSON action arrays** from screenshots. Natural language descriptions are USELESS here.

Tier 1: Best for Navigation

Model	Provider	JSON Quality	Speed	Cost/1M	Context
Gemini 2.5 Flash	Google	Excellent	800ms	$0.10	1M tokens
Kimi K2.5	Moonshot	Very Good	1000ms	$0.60	256K tokens
GPT-4o	OpenAI	Excellent	1200ms	$5.00	128K tokens
Claude Sonnet 4	Anthropic	Good	1500ms	$3.00	200K tokens

Gemini 2.5 Flash is the clear winner for navigation:

- Produces clean JSON arrays consistently
- Fastest response time (~800ms)
- Cheapest ($0.10/1M tokens)
- 1M token context window
- Native multimodal (no separate vision API)
- Free tier available (generous quota)

Kimi K2.5 is the best budget alternative:

- MIT-licensed architecture
- $0.60/1M tokens
- Native vision capabilities
- Good JSON compliance

Tier 2: Acceptable for Navigation

Model	Provider	Notes
NVIDIA Llama 3.2 90B	NVIDIA	Good but unreliable (500 errors)
GitHub Models (GPT-4o)	GitHub	Free but rate-limited
Qwen3-VL	Alibaba	~90% UI grounding accuracy
Step-GUI	Stepfun	GUI-specialized but early stage

NOT Suitable for Navigation

Model	Provider	Why
Astica Vision	Astica.AI	Returns descriptions, NOT JSON arrays
xAI Grok	xAI	Inconsistent JSON format
Local Ollama (minicpm-v)	Ollama	Too slow for interactive navigation

---

4. Vision Models for Analysis (Analyze Phase)

These models analyze screenshots for bugs and UI issues. Rich, detailed descriptions are REQUIRED.

Tier 1: Best for Analysis

Model	Provider	Description Quality	OCR	Object Detection	Cost/1M
Astica Vision 2.5	Astica.AI	Excellent	Yes	Yes	~$0.001
GPT-4o	OpenAI	Excellent	Good	Good	$5.00
Gemini 2.5 Pro	Google	Very Good	Good	Good	$2.00
Claude Sonnet 4	Anthropic	Very Good	Good	Basic	$3.00

Astica Vision 2.5 is the clear winner for analysis:

- Specialized vision API (not a general LLM doing vision)
- Native OCR with word-level bounding boxes
- Object detection with coordinates
- Face detection with age/gender estimation
- Content moderation (adult/racy/gore detection)
- Brand/logo detection
- Landmark identification
- Color extraction
- Custom prompt support via `gpt_prompt`
- Very cheap per call

Tier 2: Good for Analysis

Model	Provider	Notes
InternVL3-78B	Open Source	State-of-the-art open-source VLM, 72.2 MMMU
GLM-4.6V	ZhipuAI	Native tool use, 128K context
Pixtral	Mistral	Efficient, good for structured content

---

5. Chat Models for Planning (Learn/Plan)

Text-only models for reasoning, planning, and report generation.

Tier 1: Best for Planning

Model	Provider	Reasoning	Context	Cost/1M	Structured Output
Gemini 2.5 Pro	Google	94.3% GPQA	1M	$2.00	Excellent
Claude Sonnet 4	Anthropic	92.8% GPQA	200K	$3.00	Excellent
GPT-5.4	OpenAI	92.8% GPQA	128K	$2.50	Excellent
Gemini 2.5 Flash	Google	~88% GPQA	1M	$0.10	Very Good

Gemini 2.5 Flash offers the best value:

- 1M token context (fits entire project KB)
- $0.10/1M tokens (100x cheaper than GPT-5)
- Strong reasoning capabilities
- Fast response times
- Free tier with generous quota

Tier 2: Budget Options

Model	Provider	Notes	Cost/1M
DeepSeek Chat	DeepSeek	Excellent reasoning, very cheap	$0.14
Groq Llama 3.3 70B	Groq	Fast inference, free tier	$0.06
Cerebras Llama 3.3 70B	Cerebras	Ultra-fast inference	Free
Kimi K2.5	Moonshot	Good reasoning, cheap	$0.60

---

6. Local & Open-Source Models

For when cloud APIs are unavailable or for privacy/cost reasons.

Vision Models (for Execute/Curiosity when used locally)

Model	Size	RAM Needed	GPU Needed	Quality
MiniCPM-V 2.6	8B	8GB	6GB VRAM	Good for simple navigation
LLaVA 1.6 Mistral	7B	8GB	6GB VRAM	Acceptable, slower
LLaVA 13B	13B	16GB	8GB VRAM	Better quality, slower
InternVL2-8B	8B	8GB	6GB VRAM	Strong UI understanding
Qwen2.5-VL-7B	7B	8GB	6GB VRAM	~90% UI grounding

Chat Models (for Plan/Learn when used locally)

Model	Size	RAM	Quality
Llama 3.3 70B	70B (Q4)	40GB	Near-GPT-4 quality
Qwen3 8B	8B	8GB	Good for basic planning
DeepSeek R1 8B	8B	8GB	Strong reasoning

Distributed Inference (llama.cpp RPC)

When a single machine can't run a large model, distribute across multiple hosts:

- **Master** (GPU host): Runs `llama-server` with `--rpc worker1:port,worker2:port`
- **Workers** (CPU/GPU hosts): Run `rpc-server` contributing RAM/VRAM
- **Model**: Split across layers — each worker computes assigned layers

Tested configuration: thinker.local (RTX 3060 6GB) + amber.local (16GB RAM) = combined 22GB+ for model inference.

Important: Local models are 10-100x slower than cloud APIs for multimodal inference. Use them as fallback, not primary for interactive QA sessions.

---

7. Bridged CLI Models

Models accessible via CLI tools running on the developer's machine.

Available Bridges

CLI Tool	Models	Vision	Notes
Claude Code (`claude`)	Claude Sonnet 4, Opus 4.6	Yes	Uses OAuth token, product-restricted
Qwen Code (`qwen-coder`)	Qwen3 Coder	No	Code-focused
OpenCode/Zen (`opencode`)	Various	No	Multi-model orchestration

When to Use Bridged Models

Bridged models are useful when:

1. You have a CLI subscription but no API key
2. The CLI handles authentication (OAuth)
3. You want to use the same model the developer uses
4. Cost is handled by the CLI subscription (not per-token)

Limitations: Slower than direct API (CLI startup overhead), no streaming in some cases, session management complexity.

---

8. Specialized Vision APIs

Astica.AI Vision

API Endpoint: `https://vision.astica.ai/describe` **Authentication**: Token in request body (`tkn` field) **Features**:

- Image captioning (confidence-scored)
- Detailed GPT-powered descriptions (`caption_GPTS`)
- Object detection with bounding boxes
- Face detection (age, gender)
- Text recognition (OCR) with word-level bounding boxes
- Content moderation (adult, racy, gore scores)
- Brand/logo detection
- Landmark identification
- Color extraction (dominant + accent)
- Custom prompts via `gpt_prompt` parameter

Best for: Phase 4 (Analyze) — rich, multi-faceted image analysis **Not suitable for**: Phase 3 (Execute/Curiosity) — cannot produce JSON action arrays

Google Cloud Vision API

Alternative specialized vision API with similar capabilities but higher cost.

---

9. Cost Analysis

Per-Session Cost Estimate (50-step curiosity + 15 tests + analysis)

Configuration	Navigation Cost	Analysis Cost	Planning Cost	Total
Budget (Gemini Flash + Astica)	$0.005	$0.002	$0.001	**~$0.008**
Balanced (Gemini Flash + GPT-4o)	$0.005	$0.025	$0.005	**~$0.035**
Premium (GPT-4o + Astica + Claude)	$0.025	$0.002	$0.015	**~$0.042**
Free (Gemini free tier + Astica free)	$0.00	$0.00	$0.00	**$0.00**
Local only (Ollama)	$0.00	$0.00	$0.00	**$0.00**

Recommendation: Budget configuration ($0.008/session) provides excellent results at negligible cost.

Monthly Cost at 10 Sessions/Day

Config	Daily	Monthly
Budget	$0.08	$2.40
Balanced	$0.35	$10.50
Premium	$0.42	$12.60
Free/Local	$0.00	$0.00

---

10. Recommended Configuration

Primary (Cloud-Only, Budget-Optimized)

```
# Navigation (Execute/Curiosity) — Gemini Flash first
GEMINI_API_KEY=your_key
# Analysis (Analyze) — Astica first
ASTICA_API_KEY=your_key
# Planning (Learn/Plan) — Same Gemini (reasoning + context)
# (uses GEMINI_API_KEY above)

# Fallbacks
KIMI_API_KEY=your_key      # Budget navigation fallback
NVIDIA_API_KEY=your_key    # Free navigation fallback
```

With Local Fallback

```
# Cloud (primary)
GEMINI_API_KEY=your_key
ASTICA_API_KEY=your_key
KIMI_API_KEY=your_key

# Local (fallback when cloud unavailable)
HELIX_OLLAMA_URL=http://thinker.local:11434
HELIX_OLLAMA_MODEL=minicpm-v:8b
```

Enterprise (Maximum Quality)

```
OPENAI_API_KEY=your_key        # GPT-4o for premium navigation
ANTHROPIC_API_KEY=your_key     # Claude for planning
GEMINI_API_KEY=your_key        # Gemini for large context
ASTICA_API_KEY=your_key        # Astica for analysis
KIMI_API_KEY=your_key          # Budget fallback
```

---

11. Revolutionary Ideas

1. Multi-Model Consensus Navigation

Instead of using one model for navigation, send the same screenshot to 3 models simultaneously and use majority-vote on the action. This dramatically reduces wrong actions.

2. Progressive Model Escalation

Start with the cheapest model (Gemini Flash). If it returns empty/invalid JSON 3 times, escalate to the next tier (Kimi → GPT-4o → Claude). This minimizes cost while maximizing reliability.

3. Screenshot Diffing Before LLM

Before sending a screenshot to the LLM, diff it against the previous screenshot. If SSIM > 0.99 (nearly identical), the action had no effect — retry with a different action WITHOUT calling the LLM again. Saves 30-50% of API calls.

4. Action Replay Buffer

Record all successful action sequences (login → browse → detail → play). On subsequent sessions, try replaying known-good sequences before falling back to LLM navigation. Saves cost and improves speed.

5. Vision Model Fine-Tuning

Fine-tune a small local model (MiniCPM-V 2.6) specifically on Catalogizer screenshots + correct action pairs. After 100 sessions of data, the local model could handle 80% of navigation without cloud API calls.

6. Dual-Screen Analysis

For Android TV testing, capture BOTH the TV screen (via ADB screencap) AND the Android TV launcher state (via UI automator dump). Send both to the LLM for richer context.

7. Autonomous Bug Reproduction

When the Analyze phase finds a bug, automatically attempt to reproduce it by replaying the action sequence that led to the buggy state. If reproducible, add it to a regression test suite.

8. Cross-Device Visual Regression

Run the same test flow on multiple devices simultaneously, capture screenshots at each step, and use vision models to detect visual differences between devices (layout shifts, missing elements, scaling issues).

12. Sources

- Using Vision LLMs For UI Testing (UW CSE 503)
- UI Testing Automation with Llama 3.2 and Gemini API (Ionio)
- Vision Language Models in Mobile App Testing (Drizz)
- Top 10 Vision Language Models 2026 (DataCamp)
- Open-Source Vision Language Models (BentoML)
- Best LLMs for Vision 2026 (VisionVix)
- AI Model Benchmarks Mar 2026 (LM Council)
- AI API Pricing Comparison 2026 (IntuitionLabs)
- AI Model & API Analysis (Artificial Analysis)
- Best AI Testing Tools 2026 (Virtuoso QA)
- Best Local LLMs for 16GB VRAM (LocalLLM)
- Open-Source VLMs 2026 (Labellerr)
- Astica Vision API Documentation
- VLC Media Player Source Code (VideoLAN)